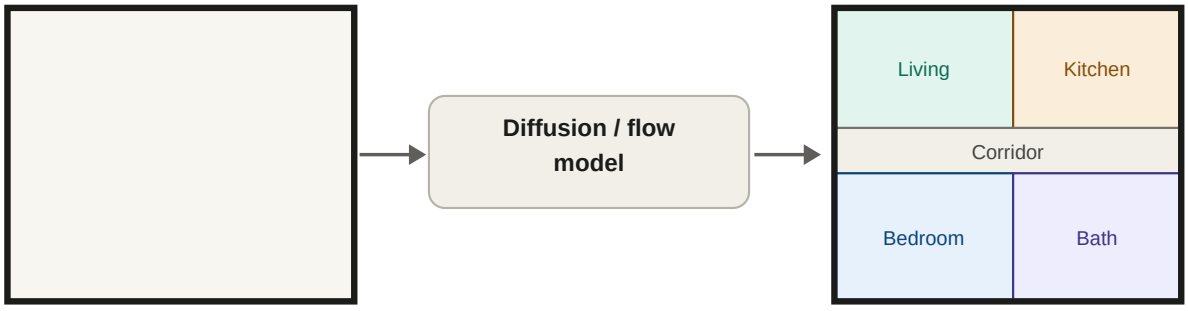
DAVIS × PARIS 2026 · CHALLENGE BRIEF

Mirror Mirror on the Wall: Who has the best room of them all?

Given only the outline of an apartment, generate a complete and plausible set of interior rooms as vector polygons, using a diffusion or flow-matching model. Entries are scored on the realism and diversity of the generated layouts, measured by FID and density and coverage against real Swiss residential floor plans.

The task

This is conditional generation. The model sees one input, the apartment outline, and must produce the rooms that fill it: where each room sits, its shape, and its type. Rooms are returned as vector polygons, never as a pixel grid. How you encode the outline, which model you build (within diffusion or flow matching), and how you parameterise rooms are all yours to decide.



From a bare outline (left), the model fills in typed rooms (right). The outline carries no rooms, walls, or graph: it is the only condition.

The data

The dataset is Modified Swiss Dwellings (MSD). The room polygons you need live in the geometry dataframe, not in the image or graph folders, which is easy to miss:

WHERE	WHAT
`mds_V2_5.372k.csv`	The room geometry. Polygons are in the **geom** column (WKT). This is the vector modality, separate from the struct_in / graph_in / graph_out / full_out folders, which this task does not use.
`entity_type == area`	Selects the room/space polygons for a given **plan_id**.
`outline`	Built from the rooms by a provided script: buffer each room out by 0.3 m, union, then buffer back in, fusing them into one exterior shell.

Input and output

Fixed. Input is the apartment outline (one polygon). Output is a set of typed room polygons. The standard outline-construction script is provided so every entry conditions on the same boundary (full script in the appendix).

Free. How you encode the outline, your model architecture (diffusion or flow matching), and how you represent rooms (rectangles, corner sequences, anything except a pixel grid).

Evaluation

Two metrics: FID and density and coverage. FID rewards realism; density and coverage reward covering the true variety of layouts and penalise mode collapse or copying the data. Scoring is run by the organisers on a held-out set of plans, with a fixed protocol (reference set, rasterisation, sample count, and seed 42), so numbers are comparable across teams.

Rules and submission

- **Time:** Saturday to Sunday.
- **Model:** diffusion- or flow-matching-based, trained from scratch.
- **Seed:** a fixed random seed of 42 throughout (data, training, sampling, evaluation).
- **Submit:** training and generation code, model weights, a generate(outline) entry point that returns room polygons, and a short methodology writeup.

Appendix: provided data-construction script

The reference script every team starts from. It loads the geometry dataframe, selects one plan's room polygons (entity_type == area), fuses them into the single outline used as the model input, and plots the input and target side by side.

```
import pandas as pd
import geopandas as gpd
from shapely import wkt
import matplotlib.pyplot as plt

# 1. Load data
csv_path = "mds_V2_5.372k.csv"
df = pd.read_csv(csv_path)
df['geom'] = df['geom'].apply(wkt.loads)
gdf = gpd.GeoDataFrame(df, geometry='geom')

# 2. Isolate a plan
sample_plan_id = 7988
apartment_gdf = gdf[gdf['plan_id'] == sample_plan_id]
rooms_gdf = apartment_gdf[apartment_gdf['entity_type'] == 'area']

# Merge the rooms to create the outline
# 1. Buffer outward by 30 centimeters (MSD uses meters as units)
# 2. Cleanly unify them into one solid geometry
# 3. Buffer back inward by 30 centimeters to restore the original scale
wall_bridge_distance = 0.3
solid_outline_geom = rooms_gdf.geometry.buffer(wall_bridge_distance).unary_union.buffer(-wall_bridge_distance)

# Convert the resulting Shapely geometry back into a GeoDataFrame for plotting
outline_gdf = gpd.GeoDataFrame(geometry=[solid_outline_geom], crs=rooms_gdf.crs)
# -------------------------------------

# 3. Plot side-by-side again
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))

# Left Plot: Condition
outline_gdf.plot(ax=ax1, facecolor='#f4f4f4', edgecolor='black', linewidth=3)
ax1.set_title("Input: Clean Apartment Outline", fontsize=14, fontweight='bold')
ax1.axis('equal')
ax1.axis('off')

# Right Plot: Target
rooms_gdf.plot(ax=ax2, cmap='Set3', edgecolor='white', linewidth=1.5)
outline_gdf.plot(ax=ax2, facecolor='none', edgecolor='black', linewidth=2, alpha=0.4)
ax2.set_title("Target: Generated Rooms", fontsize=14, fontweight='bold')
ax2.axis('equal')
ax2.axis('off')

plt.suptitle(f"Generative Task Data Pairing (Plan ID: {sample_plan_id})", fontsize=16)
plt.tight_layout()
plt.show()
```

Dataset: Modified Swiss Dwellings on Kaggle · Challenge by Davis, in partnership with the TUM.ai and Iterate.